01 - Python basics

This notebook introduces the basic building blocks of Python:

- Variables
- Numeric data
- Strings
- Indexing and slicing
- Lists
- Dictionaries

We will look at how to create each of these, and at the operations we can perform on them.

> 📝 **Note:** A few cells in the lecture notebooks are *meant* to fail. They are marked with a comment naming the error, for example `# IndexError`. Seeing the error message is the point of those cells, so run them one at a time rather than as part of a Run All.

Variables

In programming, we store the inputs and outputs of our code in *variables*.

A variable associates a **name** with a **value**. We create one by assigning a value to a name using the equality operator `=`.

In [ ]:
```
num = 10
```

Once a variable has been defined, we can access its value by calling the variable name.

In [ ]:
```
num
```

Python is a *dynamically typed* language, which means we can change the value of a variable at any point in a program. In other words, we can overwrite variables.

In [ ]:
```
num = num + 10

num
```

Naming variables

Loading [MathJax]/extensions/Safe.js

There are rules that must be followed when naming variables in Python. A variable name cannot:

- start with a number
- contain spaces
- contain special characters such as `!`, `'`, `#` or `@`

In [ ]:
```
# SyntaxError
six pack = 6
```

Instead of spaces we use underscores. This is called *snake case*, and it is the convention in Python.

In [ ]:
```
six_pack = 6
```

> ⚠️ **Warning:** Avoid using the names of Python commands as variable names. Doing so overwrites the command for the rest of the session.

In [ ]:
```
# Remove the hashtag below to overwrite the print command
# print = 6
```

> 💡 **Tip:** If you accidentally overwrite a Python command, you can get it back by restarting the notebook. Press the **Restart Kernel** button in the notebook toolbar.

Your turn

Each of the four assignments below is either invalid or a bad idea:

```
2nd_place = "silver"
exam score = 78
sum = 100
course-code = "BAN405"
```

Rewrite all four so that they run and follow the Python naming conventions. For the one that runs but is a bad idea, explain in a comment why.

In [ ]:
```

```

Numeric data

Numeric data are values containing only digits, with an optional sign and decimal point:

Loading [MathJax]/extensions/Safe.js

s (whole numbers), e.g. `10`

- **floats** (numbers with decimals), e.g. `10.0`

> 📝 **Note:** Commas are never used to write floats in Python. We use `.` for the decimal point.

We can use the `type` function to get the data type of a value.

In [ ]:
```
quantity = 10
price = 12.6

print(type(quantity))
print(type(price))
```

Note that evaluating an expression containing both integers and floats will always return a float.

In [ ]:
```
value = 7 + 4.0

value
```

Arithmetic

All the usual arithmetic operators are available:

Operator	Meaning	Example
`+` `-`	Addition, subtraction	`7 - 2`
`*` `/`	Multiplication, division	`7 / 2`
`**`	Exponentiation	`7 ** 2`
`//`	Integer division (how many whole times)	`7 // 2`
`%`	Modulo (the remainder)	`7 % 2`

Note that Python uses a *double* multiplication sign for exponentiation.

In [ ]:
```
2 ** 3
```

The last two operators are less familiar but very useful. Integer division `//` tells us how many whole times one number fits into another, and modulo `%` tells us what is left over.

In [ ]:
```
# How many whole weeks are there in 17 days, and how many days are left over
print(17 // 7)
print(17 % 7)
```

Converting between integers and floats

Loading [MathJax]/extensions/Safe.js

We can use the `int` and `float` functions to convert a value from one type to the other.

In [ ]:
```
value = 11.0

int(value)
```

However, converting a value does not change the variable. If we want to keep the change, we have to assign the result to a variable name — either a new one, or the same one again.

In [ ]:
```
# value itself is unchanged
value
```

In [ ]:
```
value = int(value)

value
```

Be careful with `int`: it does not round, it simply drops everything after the decimal point. If we want to round to the nearest whole number, we use the `round` function instead.

In [ ]:
```
print(int(12.6))
print(round(12.6))
```

Floats have limited precision

Computers store numbers in binary rather than decimal, and not every decimal number has an exact binary representation. Floats in Python carry roughly 16 digits of precision, which means calculations can accumulate very small errors.

In [ ]:
```
# Should be 0.33333333333333333333...
1/3
```

In [ ]:
```
# Should be exactly 2
total = 1/3 + 1/3 + 1/3 + 1/3 + 1/3 + 1/3

total
```

This is not a bug in Python — every programming language that uses floats behaves this way. It matters because it means you should never expect two float calculations to come out *exactly* equal.

The practical fix is to round to a sensible number of decimals when we display or compare results. The `round` function takes a second argument giving the number of decimals.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
round(total, 1)
```

> 📝 **Note:** Python uses scientific notation to display very large and very small floats.

In [ ]:
```
print(0.0000000000000005)
```

Your turn

The area of a rectangle is equal to the width multiplied by the height of the rectangle.

Assume you have a rectangle with a width of 15.2 cm and a height of 3.72 cm. Store the width and the height in variables, then calculate and display:

- the area, rounded to two decimals
- the area rounded to the nearest whole square centimeter
- the area with everything after the decimal point dropped

The last two should give different answers. Make sure you can say why.

In [ ]:
```

```

Strings

In Python, we call text data *strings*.

A string is any sequence of characters. The characters can be letters, numbers, special characters, spaces and so on.

A string is created by placing the sequence of characters between a pair of single quotes `''` or double quotes `""`. Either is fine, as long as you are consistent.

In [ ]:
```
sentence = "This is how I prefer to write strings"

sentence
```

A string can contain quotation marks, as long as they are different from the ones delimiting the string. Below, the double quotes inside the text end the string far too early, and Python cannot make sense of what is left.

In [ ]:
```
# SyntaxError
quote = "She said "hello" and walked away."
```

Loading [MathJax]/extensions/Safe.js

Delimiting the string with single quotes instead solves the problem.

In [ ]:
```
# Valid string
quote = 'She said "hello" and walked away.'

quote
```

> 💡 **Tip:** We use double quotes by default in this course, and switch to single quotes only when the text itself contains double quotes.

We can use the `print` function to display a string without the quotation marks.

In [ ]:
```
print(quote)
```

A string must normally be written on a single line. However, we can display text across several lines using the newline control character, `\n`.

In [ ]:
```
print("This is a list of numbers: \n1\n2\n3\n4")
```

We can use the `len` function to get the length of a sequence, such as a string.

In [ ]:
```
word = "Python!"

len(word)
```

Note that there is a difference between an empty string and a string containing only spaces — the spaces are characters too.

In [ ]:
```
empty_str = ""
space_str = "     "

print(len(empty_str))
print(len(space_str))
```

Converting between strings and numbers

We can use the `str` function to convert a number to a string, and the `int` and `float` functions to convert a string containing only digits back to a number.

In [ ]:
```
num = 10

str(num)
```

In [ ]:
```
str_float = "10.2"

float(str_float)
```

String methods

Loading [MathJax]/extensions/Safe.js

Strings come with a set of built-in *methods* — functions that we call on the string itself using a dot. Here are four of the most useful.

In [ ]:
```
messy = "   bergen   "

print(messy.upper())
print(messy.lower())
print(messy.strip())
print(messy.replace("bergen", "oslo"))
```

Strings are an **immutable** data type. That means an operation on a string never alters the string itself — it produces a *new* string and leaves the original untouched.

In [ ]:
```
word = "Python!"

word.upper()
```

In [ ]:
```
# The original string is unchanged
print(word)
```

So if we want to keep the result, we have to assign it to a variable name — either a new one, or the same one again. This will be a recurring theme.

In [ ]:
```
word = word.upper()

print(word)
```

Combining strings

We can use the `*` operator to repeat a string, which is a quick way to draw a separator.

In [ ]:
```
print("*" * 30)
print("Welcome to BAN405!")
print("*" * 30)
```

And we can use the `+` operator to join, or *concatenate*, strings together.

In [ ]:
```
word1 = "Hello"
word2 = "world!"

print(word1 + " " + word2)
```

To include a number in a string this way, we first have to convert it with `str`, otherwise Python complains that it cannot add a string and a number.

In [ ]:
```
temp = 15.4

print("Today's temperature is " + str(temp) + " degrees.")
```

Loading [MathJax]/extensions/Safe.js

f-strings

Building output with `+` and `str` works, but it becomes unreadable very quickly. A much better way is an **f-string**: put the letter `f` immediately before the opening quote, and then write variables directly inside curly brackets `{}`.

In [ ]:
```
temp = 15.4
city = "Bergen"

print(f"Today's temperature in {city} is {temp} degrees.")
```

We can also control *how* a value is displayed by writing a format specification after a colon. This is the neatest way to handle the rounding issues we saw earlier.

In [ ]:
```
value = 1/3

print(f"Unformatted:  {value}")
print(f"Two decimals: {value:.2f}")
print(f"No decimals:  {value:.0f}")
print(f"Percentage:   {value:.1%}")
```

A comma in the format specification adds a thousands separator, which makes large numbers readable.

In [ ]:
```
population = 5550203

print(f"Norway has {population:,} inhabitants.")
```

> 💡 **Tip:** f-strings are the recommended way to build output in Python. We will use them constantly.

Your turn

Store your first name in a variable called `name`, then display:

- your name in capital letters
- a sentence, built with an *f-string*, stating how many letters your name contains — for example "My name is ... and it contains ... letters."

In [ ]:
```

```

Indexing and slicing

A string is a *sequence* of characters, and Python gives us a single operator for pulling pieces out of any sequence: the index operator `[]`.

Loading [MathJax]/extensions/Safe.js

Indexing

We select a single character by placing its position inside `[]`:

```
string_name[index]
```

> 📝 **Note:** Python uses **zero-based indexing**. The first item in a sequence is at index 0, not 1.

In [ ]:
```
greeting = "Hello!"

greeting[0]
```

In [ ]:
```
greeting[1]
```

This means the last valid index is the length of the sequence *minus one*. Asking for anything beyond that is an error.

In [ ]:
```
len(greeting)
```

In [ ]:
```
# IndexError
greeting[6]
```

In [ ]:
```
greeting[len(greeting) - 1]
```

Counting to the end of a long string is tedious, so Python lets us use **negative indices**, which are interpreted as positions counted from the end. `-1` is the last character.

In [ ]:
```
print(greeting[-1])
print(greeting[-2])
```

Slicing

We can also use `[]` to extract a whole *range* of characters, which is called **slicing**:

```
string_name[start:stop]
```

Slicing extracts the characters from `start` up to but **not including** `stop`.

In [ ]:
```
sentence = "This is an intro class to Python"
```

In [ ]:
```
# Extract the first word
sentence[0:4]
```

Loading [MathJax]/extensions/Safe.js

Excluding the stop index looks strange at first, but it has a convenient consequence: the length of the slice is always `stop - start`. So `sentence[0:4]` gives exactly 4 characters, and two slices written as `[0:4]` and `[4:8]` fit together with no overlap and no gap.

In [ ]:
```
sentence[4:8]
```

We can leave out either index. If we omit the start, Python slices from the beginning of the sequence; if we omit the stop, it slices to the end.

In [ ]:
```
# From the beginning
sentence[:4]
```

In [ ]:
```
# To the end
sentence[26:]
```

Negative indices work in slices too, which is the easiest way to take something off the end.

In [ ]:
```
# The last six characters
sentence[-6:]
```

> 💡 **Tip:** Slicing is more forgiving than indexing. `greeting[6]` raises an `IndexError`, but a slice that runs past the end of the sequence simply stops at the end and returns what it can.

In [ ]:
```
greeting[2:100]
```

A third index: the step

The index operator actually takes a third, optional value — the **step**, which says how many positions to move each time:

```
string_name[start:stop:step]
```

In [ ]:
```
digits = "0123456789"

# Every second character
digits[0:10:2]
```

Since the start and stop can be omitted, a step on its own is often all we need.

In [ ]:
```
# Every third character
digits[::3]
```

Loading [MathJax]/extensions/Safe.js

A negative step walks backwards, so omitting start and stop and stepping by `-1` reverses a sequence.

In [ ]:
```
sentence[::-1]
```

Your turn

A product code is stored in the variable below:

```
code = "NO-2026-BAN405-0147"
```

Using indexing and slicing only, extract and display:

- the country code (the first two characters)
- the year
- the course code
- the running number (the last four characters — use a negative index)
- the running number written backwards

In [ ]:
```

```

Lists

Lists in Python are very similar to our everyday concept of lists. In the real world we read off, add and cross off items in lists, and we can do the same with lists in Python.

Lists are one of the most common ways to store data in Python, and they are very useful when we want to store a collection of items in a single variable.

We create a list by placing a comma-separated sequence of items inside square brackets:

```
lst_name = [item1, item2, item3, ...]
```

In [ ]:
```
fruits = ["banana", "apple", "cherry"]

fruits
```

In [ ]:
```
print(type(fruits))
print(len(fruits))
```

A list can contain any Python data type, and the items do not all have to be the same type.

In [ ]:
```
mixed_lst = ["banana", 2, 42.5, 10*2]
```

Loading [MathJax]/extensions/Safe.js

```
mixed_lst
```

Indexing and slicing lists

A list is a sequence, so the index operator works exactly as it did for strings. Same zero-based indexing, same negative indices, same slicing rules.

In [ ]:
```
print(mixed_lst[0])
print(mixed_lst[-1])
print(mixed_lst[1:3])
```

> 📝 **Note:** Slicing a list always returns a *list*, even when the slice contains a single item.

In [ ]:
```
mixed_lst[0:1]
```

Your turn

Store the names of the weekdays (Mon-Fri) in a list called `day_lst`, then display the last day in the list in *three* different ways.

In [ ]:
```

```

Operations on lists

Unlike numbers and strings, lists are **mutable**: we can change a list after it has been created, without assigning the result to a new variable.

We add a new item to the end of a list using `append`.

In [ ]:
```
fruits
```

In [ ]:
```
fruits.append("pear")
```

In [ ]:
```
fruits
```

Note what just happened: the `append` cell produced no output at all, and yet `fruits` changed. That is what "mutable" means in practice, and it is the opposite of how the string methods behaved.

Some other useful list methods:

In [ ]:
```
numbers = [5, 3, 9, 1]

sert(0, 7)    # insert 7 at position 0
print(numbers)
```

Loading [MathJax]/extensions/Safe.js

```
numbers.remove(9)       # remove the first occurrence of the value 9
print(numbers)

numbers.pop()           # remove and return the last item
print(numbers)
```

We can also sort a list. There are two ways to do this, and the difference between them matters: `sort` is a method that reorders the list **in place**, while `sorted` is a function that leaves the original alone and returns a **new** sorted list.

In [ ]:
```
print(sorted(numbers))    # a new, sorted list
print(numbers)            # the original is untouched
```

In [ ]:
```
numbers.sort()            # reorders the list itself

print(numbers)
```

Instead of appending, we can use the `+` operator to combine several lists into a single list. Because this creates a new list rather than modifying an existing one, we have to store the result.

In [ ]:
```
vegetables = ["potato", "carrot", "broccoli", "onion"]

new_lst = fruits + vegetables

new_lst
```

When a list contains only numbers, we can apply `sum`, `min` and `max` to it.

In [ ]:
```
num_lst = [1, 2, 3, 4, 5, 6, 7]

print(sum(num_lst))
print(min(num_lst))
print(max(num_lst))
```

> ⚠️ **Warning:** Never name a list `list`, and never name a dictionary `dict` — these are already the names of built-in Python functions.

> 📝 **Note:** Python also has *tuples*, created with parentheses `()` instead of square brackets, e.g. `("banana", "apple", "cherry")`. Tuples work like lists but are immutable — they cannot be changed after they are created.

In [ ]:
```

```

Loading [MathJax]/extensions/Safe.js

Nested lists

A list can contain other lists. This is known as a *nested* list, and it is how we represent a table before we have anything better.

In [ ]:
```
foods = [fruits, vegetables]

foods
```

In [ ]:
```
# The first list
foods[0]
```

In [ ]:
```
# The second item in the first list
foods[0][1]
```

Your turn

Consider the nested list `table`, where each sublist contains the test scores for a single student on three different tests:

```
table = [
    [85, 91, 89],  # test scores for student 1
    [78, 81, 86],  # test scores for student 2
    [62, 75, 77],  # test scores for student 3
    [70, 65, 72]   # test scores for student 4
]
```

Use the list to calculate and display the average test score for the **second student**.

In [ ]:
```

```

Names and objects

We have now seen strings behave one way and lists another, and it is worth a few minutes to understand *why*, because it explains a whole family of bugs.

A variable in Python is not a box holding a value. It is a **name attached to an object**. Assignment attaches a name to an object — it does not make a copy of it.

So when we write `lst_b = lst_a`, we do not get a second list. We get a second name for the same list.

In [ ]:
```
lst_a = ["banana", "apple"]
lst_b = lst_a               # lst_b is another name for the SAME list

lst_b.append("cherry")
```

Loading [MathJax]/extensions/Safe.js

```
print(lst_a)
print(lst_b)
```

We only appended to `lst_b`, but `lst_a` changed as well — because there was only ever one list.

This never happens with strings or numbers, because they are immutable. No string method can modify a string in place, so the second name always ends up pointing at a new object.

In [ ]:
```
str_a = "banana"
str_b = str_a

str_b = str_b.upper()       # creates a NEW string

print(str_a)
print(str_b)
```

If we want a genuinely independent copy of a list, we have to ask for one. Taking a full slice is one way to do it, since we know that slicing returns a new list.

In [ ]:
```
lst_a = ["banana", "apple"]
lst_b = lst_a[:]            # a new list with the same contents

lst_b.append("cherry")

print(lst_a)
print(lst_b)
```

There is a second trap hiding in the same idea. A method that changes a list in place has nothing useful to give back, so it returns `None` — Python's way of saying "no value". Assigning that return value therefore destroys the list.

In [ ]:
```
fruits = ["banana", "apple", "cherry"]

fruits = fruits.sort()      # DON'T do this

print(fruits)
```

> ⚠ **Warning:** The rule of thumb is: if a method changes an object *in place*, call it on its own (`fruits.sort()`). If a function returns a *new* object, assign it (`fruits = sorted(fruits)`). Writing `fruits = fruits.sort()` silently sets `fruits` to `None`.

Your turn

Consider the following code:

Loading [MathJax]/extensions/Safe.js

```
prices = [199, 249, 149]
budget_prices = prices
budget_prices.append(99)
```

Before running anything, predict what `prices` will contain. Then run the code to check.

Finally, rewrite the code so that `budget_prices` gets the extra item but `prices` is left unchanged.

In [ ]:

Dictionaries

A dictionary is another data type used to store several items in a single variable. Dictionaries in Python are like dictionaries in the real world: we look up a word (the **key**) and read off its definition (the **value**).

Unlike a list, each item in a dictionary is associated with a key rather than a position. We create a dictionary by placing comma-separated *key-value* pairs inside curly brackets `{}`. Each pair is written as the key, then a colon `:`, then the value:

```
dict_name = {key1: value1, key2: value2, key3: value3, ...}
```

To make a dictionary readable, we usually write each key-value pair on its own line.

In [ ]:
```
student = {
    "name"      : "Anne Smith",
    "student_no": "s1234",
    "course"    : "BAN405",
    "score"     : 82
}

student
```

Like a list, a dictionary has a length — the number of key-value pairs it contains.

In [ ]:
```
len(student)
```

But unlike a list, a dictionary has no positions, so we cannot select an item using a numeric index.

In [ ]:
```
# KeyError
student[0]
```

Instead, we use the key to look up a value. We can use the `keys` method to see which lable, and the `values` method to see the values.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
student.keys()
```

In [ ]:
```
student.values()
```

In [ ]:
```
print(student["name"])
print(student["score"])
```

> 💡 **Tip:** Dictionary keys are case-sensitive, and a misspelled key raises a `KeyError`. If you get one, print `.keys()` first and check what is actually there.

Your turn

The table below records the daily temperature from Monday to Friday.

Day	Temperature (°C)
Monday	22.5
Tuesday	24.0
Wednesday	19.8
Thursday	21.3
Friday	23.1

Store the data in a dictionary called `days`, then use the dictionary to display:

- the temperature on Wednesday
- the average temperature across the week, rounded to one decimal

In [ ]:
```
```

Like lists, dictionaries are mutable. We can update the value of an existing key, and we can add entirely new key-value pairs.

In [ ]:
```
# Update the value of an existing key
student["score"] = 92

# Add a new key-value pair
student["university"] = "NHH"

student
```

The values in a dictionary can be of any type, including lists — which is how we store several observations under a single key.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
student["score"] = [92, 87, 82]

student
```

And we can then index into that list, exactly as we did with nested lists.

In [ ]:
```
# The second score
student["score"][1]
```

Additional resources

- The Python Tutorial — the official introduction, sections 3 and 5 cover this notebook's material
- Built-in functions — the full list of functions such as `len`, `sum`, `round` and `type`
- Automate the Boring Stuff with Python — chapter 1, free to read online
- String formatting with f-strings — all the format specifications
- SKL401 — lessons 1.7, 1.8 and 1.9 cover the same ground more briefly, if you would like a second explanation

Next week: decisions — comparing values, and getting a program to choose between different courses of action.

Loading [MathJax]/extensions/Safe.js